

(https://
profile.intra.42.fr)

SCALE FOR PROJECT THE ANSWER PROTOCOL (/PROJECTS/THE-ANSWER-PROTOCOL)

You should evaluate 2 students in this team



Git repository

git@vogsphere.42london.com:vogsphere/intra-uuid-40ad3d35-e6f9-45bf-9 

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.
- Identify with the person (or the group) evaluated the eventual dysfunctions of the work. Take the time to discuss and debate the problems you have identified.
- You must consider that there might be some difference in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade him/her as honestly as possible. The pedagogy is valid only and only if peer-evaluation is conducted seriously.

Guidelines

Please respect the following rules:

- Only grade what is in the group's Git repository (this is a 2-learner group project).
- The evaluation stops immediately if mandatory files are missing or extra unauthorized files are present.
- The project must strictly follow the attached protocol; all commands and events described there must be implemented.
- Both a CLI and a GUI client must be present and functional.
- The server, clients, and build system must work on a clean environment with only allowed dependencies.
- The project must be implemented in one of the following languages: Rust, Go, C++ (any version), Java, or C#. Python is explicitly forbidden.
- Verify that both group members have meaningful contributions in the Git history.

Attachments

 subject.pdf (https://cdn.intra.42.fr/pdf/pdf/202748/en.subject.pdf)

 rfc.tar.gz (https://cdn.intra.42.fr/document/document/48166/rfc.tar.gz)

Preliminaries

Content

Check that the repository contains all mandatory deliverables:

- Building tool (Makefile or equivalent) at the root with required targets
- TCP server implementation
- CLI client
- GUI client
- Static world data (YAML or JSON)
- README.md file at the root Also verify that no unauthorized files are present. If any of these checks fail, stop the review here.

✓ Yes

✗ No

README Documentation

Verify the README.md file contains all required sections:

- First line in italics: "This project has been created as part of the 42 curriculum by <login1>, <login2>."
- Description section clearly presenting the project goal and overview
- Instructions section with compilation, installation, and execution information
- Resources section listing references and describing AI usage (if any)
- Architecture section explaining server design choices
- Protocol Implementation section documenting any RFC deviations
- Combat System section describing combat mechanics and design choices
- Quest System section explaining quest progression and implementation
- World Design section describing world layout and NPC/item distribution
- Server Logging section documenting logging implementation and monitoring capabilities
- Group Contributions section indicating each member's responsibilities
- Building and Running section with detailed instructions
- Testing section explaining how to test functionality

✓ Yes

✗ No

Mandatory Part

World requirements

Verify world structure meets minimum requirements:

- At least 8 interconnected rooms forming loops with at least one optional branch
- At least 3 distinct NPC roles: dialogue NPCs, quest-giver NPCs, and enemy NPCs
- At least 4 distinct items with at least 2 obtainable in-world
- At least 2 implemented quests of different types
- Movement allows full circuit exploration (no "line-only" maps)
- All NPCs and items referenced in rooms are properly defined in world data

✓ Yes

✗ No

Build

Verify the project is implemented in one of the allowed languages:

Rust, Go, C++ (any version), Java, or C#. If Python is used, fail the review immediately.

Run the install target from the building tool on a clean environment and verify it completes without errors.
Ensure only appropriate dependencies for the chosen language are installed (networking libraries, GUI toolkit, data parsers such as YAML/JSON parsers).
If build fails or Python is detected, stop the review here.

☒ Yes☐ No

Protocol Compliance

Start the server and connect with both the CLI and GUI clients.
Verify that the greeting matches RFC 42TAP specifications.
Execute every command and event defined in the RFC document to confirm they behave as specified.
Verify that all message formats follow the ABNF syntax definitions in the RFC.
Send malformed or invalid commands and ensure the server returns the correct ERR response without crashing.
Check that error codes match exactly those specified in Section 7.2 of the RFC.

☒ Yes☐ No

Server Behaviour

Confirm the server loads the world data and validates exits and references.
Check that room presence and chat events are broadcast only to the intended recipients.
Disconnect a client while an event is being broadcast and ensure the server removes the client cleanly and continues operating.

☒ Yes☐ No

CLI Client

Connect using the CLI client and verify:

- Commands can be sent interactively
- Responses are displayed immediately
- Asynchronous events (chat, presence) appear while waiting for input Test at least one full flow: CONNECT → LOOK → MOVE → CHAT → WHO → QUIT.

☒ Yes☐ No

GUI Client

Connect using the GUI client and verify:

- Room details, items, NPCs, and exits are displayed
- Chat is separated by scope (Global, Room, Group)
- Buttons for actions send the correct commands
- Player counts in the room and on the server update in real time Ensure the GUI remains responsive while receiving events.

☒ Yes☐ No

Robustness

Test the server and clients under edge cases:

- Disconnect a client abruptly; server must continue and remove the session
- Send multiple commands quickly from different clients; responses remain correct
- Attempt simultaneous MOVE actions from different clients and check for correct presence events
- Create and disband groups while members join/leave quickly; group state remains consistent

☒ Yes

☐ No

Network Features

Check that the server correctly handles:

- Multiple commands in a single TCP packet
- Commands split across packets
- Unicode characters in usernames or messages without encoding errors
- Control characters in messages are either rejected or safely handled

☒ Yes

☐ No

Network Feature

Test the inventory and NPC interaction features:

- TAKE command to pick up items from rooms
- DROP command to drop items from inventory
- INVENTORY command to list player's items
- TALK command to interact with NPCs
- ATTACK command to initiate combat with enemy NPCs
- STATUS command to check player health and combat status
- QUEST command to request quests from quest-giver NPCs
- QUESTS command to list active and completed quests Verify that items, NPCs, combat mechanics, and quest system are properly managed and responses are correct.

☒ Yes

☐ No

Data Integrity

Verify that LOOK outputs are valid JSON, contain consistent IDs, and match the current game state.

Ensure all items and NPCs in rooms are defined in the world data.

Moving between rooms and back should result in consistent and accurate LOOK data.

☒ Yes

☐ No

Dynamic Item Management

Test the dynamic item system thoroughly:

- Take an item from a room and verify it disappears from LOOK output
- Attempt to take the same item again and confirm it returns ITEM_NOT_FOUND error
- Have a second player try to take the same item and verify they cannot
- Drop the item and verify it reappears in the room for other players
- Test both item IDs (e.g., "item.herbs") and display names (e.g., "Herbs")
- Verify multi-word item names work correctly (e.g., "Loaf of Bread")

☒ Yes☐ No

Combat System

Test the combat system implementation:

- Verify players start with 100 HP using STATUS command
- Use ATTACK command on enemy NPCs and confirm damage is dealt
- Check that enemy NPCs can counter-attack and reduce player HP
- Verify that players with 0 HP respawn at a safe location with reduced health
- Test that non-hostile NPCs cannot be attacked (should return NPC_NOT_HOSTILE error)
- Confirm combat results are logged and broadcast to relevant players
- Verify the group's combat mechanics design is documented in README with clear justification

☒ Yes☐ No

Quest System

Test the quest system implementation:

- Use QUEST command on quest-giver NPCs to receive quests
- Use QUESTS command to list active and completed quests
- Verify at least 2 different quest types are implemented (fetch item, defeat NPC, deliver item)
- Test quest completion validation and reward systems
- Confirm NPCs without quests return NO_QUEST_AVAILABLE error
- Verify the group's quest progression mechanics are documented in README with implementation approach

☒ Yes☐ No

Server Logging

Verify comprehensive server logging implementation (MANDATORY):

- Check that all client connections and disconnections are logged with timestamps and IP addresses
- Verify every command received from clients is logged with player name and parameters
- Confirm all server responses and error codes sent to clients are logged
- Test that world state changes (item movements, NPC interactions, combat results) are logged
- Verify quest progress and completion events are logged
- Check that structured logging format (JSON recommended) is used for easy parsing
- Confirm log levels (INFO, WARN, ERROR) are included for different event types
- Test that potential abuse patterns (command flooding, rapid connections) are monitored and logged
- Verify all logs include precise timestamps and are written to appropriate output streams
- Check that logging does not significantly impact server performance or responsiveness

☒ Yes☐ No

Recode Exercise

Request a brief modification to verify understanding of the project:

- Ask the group to make a small change to one of the systems (e.g., modify NPC dialogue, adjust combat damage, add a simple quest step)
- The modification should be feasible within a few minutes
- Verify that both group members understand the codebase and can explain their implementation choices
- Confirm the modification works correctly and doesn't break existing functionality This step verifies actual understanding of the project implementation.

✔ Yes

✕ No

Ratings

Don't forget to check the flag corresponding to the defense

✔ Ok

★ Outstanding project

Empty work

📌 Incomplete work

⚙️ Invalid compilation

📄 Cheat

💥 Crash

👥 Incomplete group

⚠️ Concerning situation

💧 Leaks

💬 Can't support / explain code

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General
Terms of Use
([https://
profile.intra.42.fr/
legal/terms/33](https://profile.intra.42.fr/legal/terms/33))

Declaration on
the use of
cookies ([https://
profile.intra.42.fr/
legal/terms/2](https://profile.intra.42.fr/legal/terms/2))

Privacy policy
([https://
profile.intra.42.fr/
legal/terms/5](https://profile.intra.42.fr/legal/terms/5))

General term of
use of the site
([https://
profile.intra.42.fr/
legal/terms/6](https://profile.intra.42.fr/legal/terms/6))

Rules of
procedure
([https://
profile.intra.42.fr/
legal/terms/4](https://profile.intra.42.fr/legal/terms/4))

Terms of use for
video
surveillance
([https://
profile.intra.42.fr/
legal/terms/1](https://profile.intra.42.fr/legal/terms/1))

Legal notices
([https://
profile.intra.42.fr/
legal/terms/3](https://profile.intra.42.fr/legal/terms/3))